
차세대 자동차관리정보시스템 연계 호출 및 개발 가이드

2024. 05.

- 목차 -

1. 개요	1
2. 표준 연계 방식별 데이터 정의	4
3. 표준 암호·복호화 및 전자서명 절차	9
4. 연계 개발 샘플	13

1. 개요

본 문서는 한국교통안전공단 차세대 자동차관리정보시스템 서비스 사용을 위한 연계 인터페이스에 대한 호출 및 개발 가이드이다.

□ 연계 서비스 신청

신규 연계의 경우, 국토교통부에서 정보 제공에 대해 승인이 완료되면 연계호출 URL, API-KEY, 인증서(한국교통안전공단 공개키, 이용기관 개인키)가 발급된다.

□ SERVICE URL

한국교통안전공단 차세대 자동차관리정보시스템 서비스를 사용하기 위한 호출 정보

- 개발
 - RESTful, File: <http://210.91.35.175:28080/hub/kotsa>
 - SOAP: <http://210.91.35.175:28080/hub/ws/kotsa>
- 운영
 - 추후 안내(2024년 11월 예정)

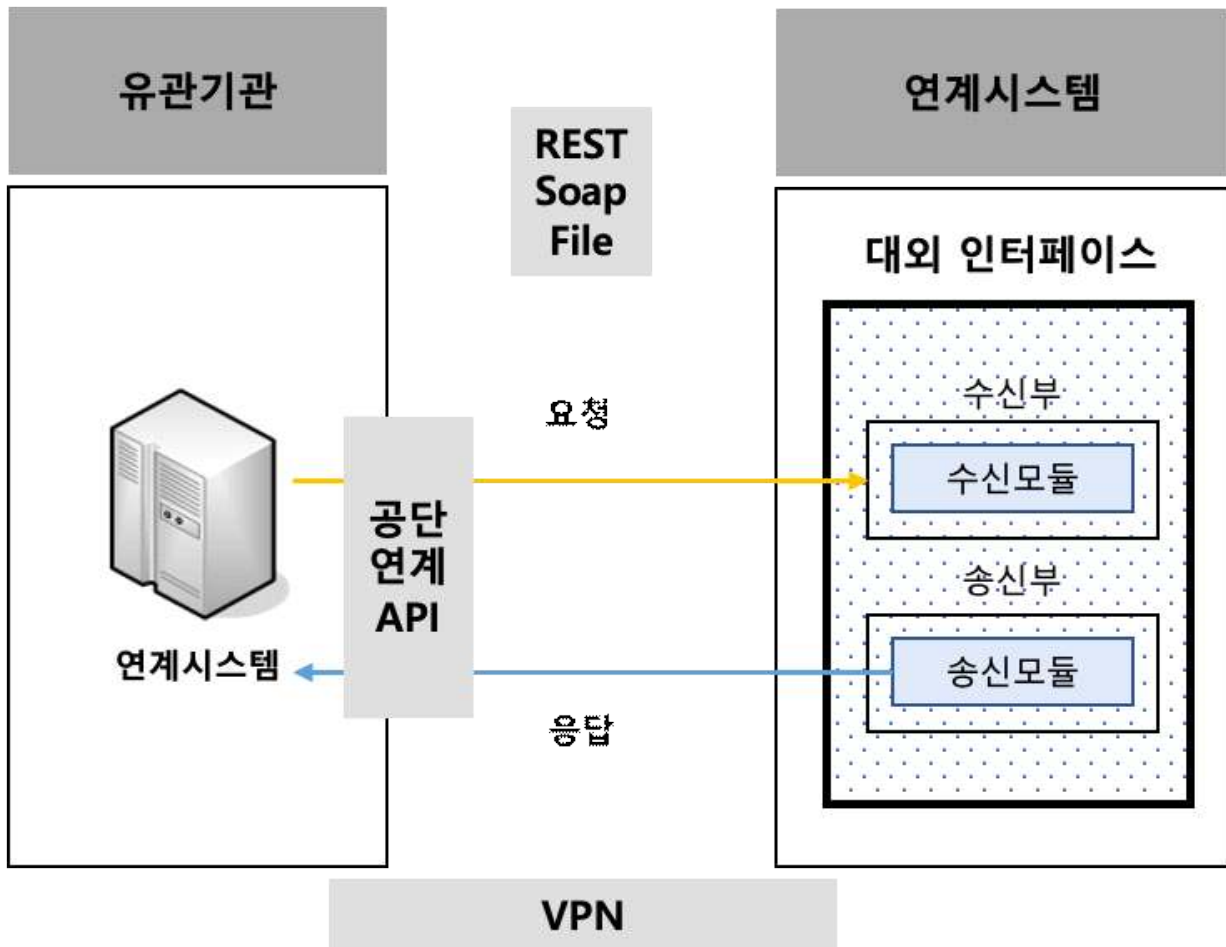
※ 상세 입/출력 전문 정보와 API-KEY, 테스트용 인증서는 추후 차세대 자동차관리정보시스템 사업단에서 서비스 별로 전달 예정

□ 차세대 자동차관리정보시스템 표준연계서비스 메시지 규격

○ 메시지 형식 개요

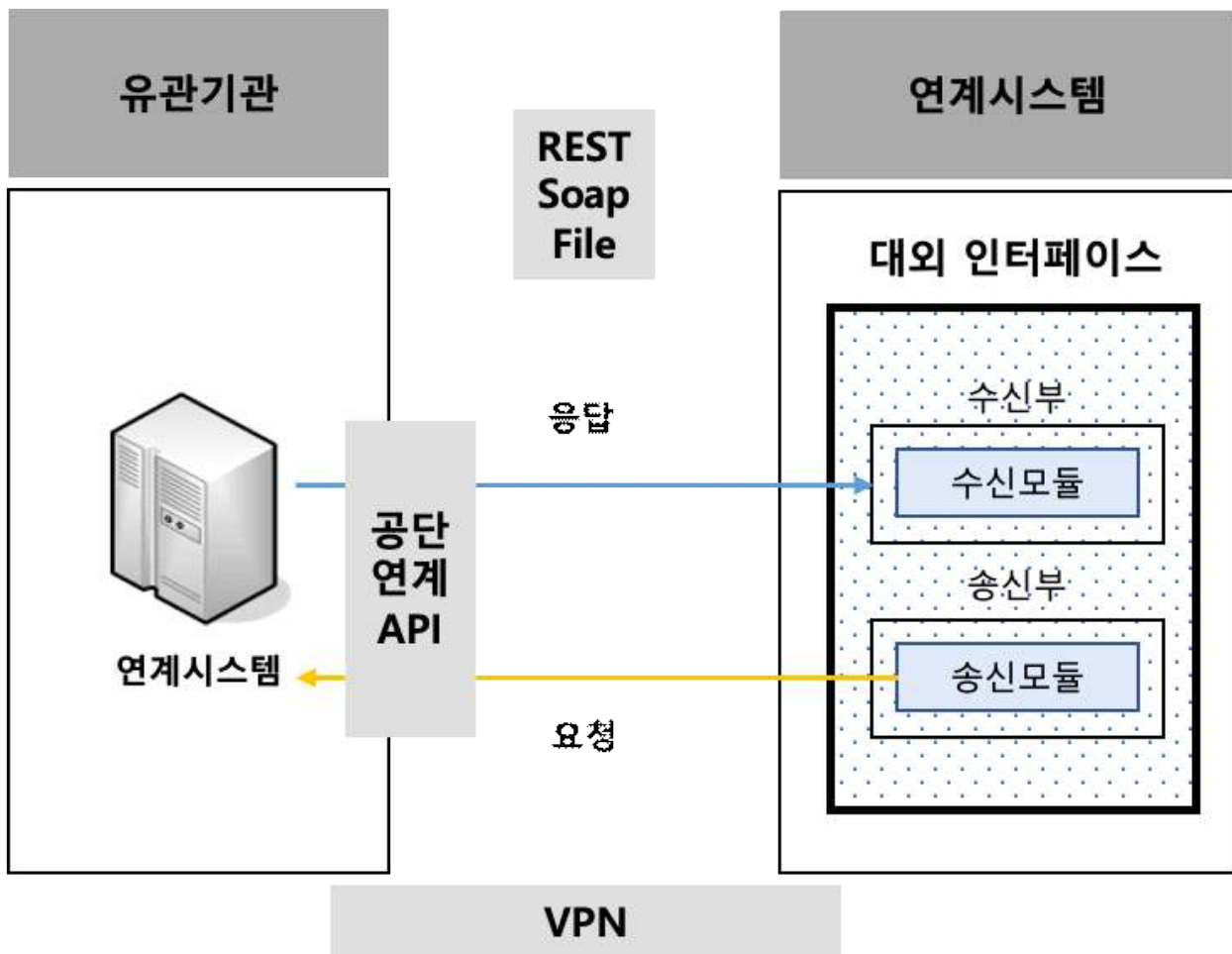
- 차세대 자동차관리정보시스템 표준 연계 방식은 RESTful, SOAP, File 방식을 사용
- RESTful 방식은 JSON 메시지를 이용하여 전문을 송·수신 함
- SOAP 방식은 XML 형태의 SOAP 1.2 규격에 맞춰 전문을 송·수신 함
- File 방식은 HTTP Multipart 형식으로 송·수신 함
 - 이용기관에서 차세대 자동차관리정보시스템으로 파일을 전송할 경우는 Multipart 형식으로 전송
 - 이용기관에서 파일을 수신할 경우는 HTTP Multipart 형식으로 수신할 수 있는 정보 제공 필요

- **유관기관 수신 연계**(유관기관이 한국교통안전공단으로부터 정보를 제공 받음)



- 유관기관과 한국교통안전공단은 VPN망 으로만 연결되어 있다.
- 유관기관 연계시스템은 요청이 필요한 정보가 있으면 전문을 구성하여 한국교통안전공단에서 배포한 공단 연계 API를 이용하여 한국교통안전공단 연계시스템에 요청 정보를 전송한다.
- 한국교통안전공단 연계시스템은 요청내용을 수신하고 전문 분석후 요청한 정보를 조회하여 응답 전문을 구성후 요청한 유관기관 연계 시스템에 전송한다.
- 파일방식 송신일 경우 응답 내용이 없을 수 있다.

○ **유관기관 송신 연계**(유관기관이 한국교통안전공단에 정보를 제공함)



- 유관기관과 한국교통안전공단은 VPN망 으로만 연결되어 있다.
- 한국교통안전공단 연계시스템은 필요한 정보가 있으면 전문을 구성하여 유관기관 연계시스템에 요청 정보를 송신한다.
- 유관기관 연계시스템은 한국교통안전공단의 요청내용을 수신하고 전문 분석 후 요청한 정보를 조회하여 응답 전문을 구성후 요청한 한국교통안전공단 연계 시스템에 응답한다.
- 파일방식 수신일 경우 응답 내용이 없을 수 있다.

2. 표준 연계 방식별 데이터 정의

□ RESTful 데이터 정의

○ RESTful 포맷 정의

- RESTful 의 데이터는 JSON 형식을 사용
- POST method 만 이용
- HTTP Header의 Accept의 값과 Content-Type의 값은 동일 하게 기재(application/json)
- 문자셋(charset)은 UTF-8을 사용(문자셋이 다를 경우 사전 협의 필요)

○ HTTP Header 정보

- HTTP protocol의 Header에 해당 값들을 정의한다.

키	데이터 형식	필수 여부	예시	설명
Accept	string	o	"application/json"	Content-Type의 값과 동일하게 기재
Content-Type	string	o	"application/json"	본문의 미디어 타입 기재
cvmis_apikey	string	o	"C919D8A2-61F67E56-12312312-76665842"	연계 서비스가 승인 시 발급되는 api key 사용

- HTTP Header 예제

Accept : application/json	//Content-Type과 동일
cvmis_apikey : C919D8A2-61F67E56-12312312-76665842	//한국교통안전공단에서 발급해주는 api key
Content-Type : application/json	//데이터 포맷이 JSON

○ Body 정보

- Body에 해당 값들을 정의한다.

키	데이터 형식	필수 여부	예시	설명
data	string	o	{"data": }	data 내 항목은 연계별로 상이함

- Body 예제(HTTP Body의 전체 데이터에 대해 암호화 및 전자서명을 적용한다.)
 - 암호화 전 Body

```
{
  "data":[
    {
      "IHIDNUM":"9104251*****",
      "NM":"홍길동"
    }
  ]
}
```

```
}  
]  
}
```

· 암호화 후 Body

MIIKIAIBATENMA\$GCWCGSAFIawQCATCCAkAGCSq...

※ HTTP Body의 전체 데이터를 암호화 한다.

□ SOAP 데이터 정의

○ SOAP 포맷 정의

- SOAP 은 HTTP 프로토콜의 SOAP 1.2 버전을 사용
- POST method 만 이용
- HTTP Header의 Accept의 값과 Content-Type의 값은 동일 하게 기재(application/xml)
- 문자셋(charset)은 UTF-8을 사용(문자셋이 다를 경우 사전 협의 필요)
- SOAP 포맷과 관련된 WSDL 은 사전 공유 필요
 - > 유관기관 수신 연계(유관기관이 한국교통안전공단으로부터 정보를 제공 받음)의 경우에는 한국교통안전공단에서 유관기관으로 WSDL 제공
 - > 유관기관 송신 연계(유관기관이 한국교통안전공단에 정보를 제공함)의 경우에는 유관기관이 한국교통안전공단에 WSDL 제공 필요

○ HTTP Header 정보

- HTTP protocol의 Header에 해당 값들을 정의한다.

키	데이터 형식	필수 여부	예시	설명
Accept	string	o	"application/xml"	Content-Type의 값과 동일하게 기재
Content-Type	string	o	"application/xml"	본문의 미디어 타입 기재
cvmis_apikey	string	o	"C919D8A2-61F67E56-12312312-7665842"	연계 서비스가 승인 시 발급되는 api key 사용

- HTTP Header 예제

Accept : application/xml	//Content-Type과 동일
cvmis_apikey : C919D8A2-61F67E56-12312312-7665842	//한국교통안전공단에서 발급해주는 api key
Content-Type : application/xml	//데이터 포맷이 XML

○ Body 정보

- Body에 해당 값들을 정의한다.

키	데이터 형식	필수 여부	예시	설명
SOAP Envelope	string	o	<?xml version="1.0" encoding="UTF-8"?> <Envelope xmlns="http://schemas.xmlsoap.org/soap/envelope/"> </Envelope>	SOAP 전문 양식
Body	string	o	<Body> <function> <ABC...> </ABC...> </function> </Body>	function명 및 function 내 항목은 연계별로 상이함

- Body 예제(Body 태그 안쪽 두 번째 태그부터 메시지 전체에 대해 암호화 및 전자서명을 적용한다.)
 - 암호화 전 Body

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <getVoucherYnResponse xmlns="http://car.go.kr/link/vhcle/VoucherYn">
      <VOUCH_SPORT_AT>Y</VOUCH_SPORT_AT>
      <CNTC_RESULT_CODE>00</CNTC_RESULT_CODE>
      <CNTC_RESULT_DTLS>성공</CNTC_RESULT_DTLS>
    </getVoucherYnResponse>
  </soap:Body>
</soap:Envelope>
```

- 암호화 후 Body

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <getVoucherYnResponse xmlns="http://car.go.kr/link/vhcle/VoucherYn">
      MIICOGIBADGCAYQwggGAAGeAMGgwUDELMAkGA1...
    </getVoucherYnResponse>
  </soap:Body>
</soap:Evelope>
```

※ SOAP Function 의 값이 암호화 되어 있다.

□ File 데이터 정의

○ File 포맷 정의

- File은 HTTP Multipart 형식으로 송·수신
- POST method 만 이용
- HTTP Header의 Accept의 값과 Content-Type의 값은 동일하게 기재(multipart/form-data)
- 파일 포맷은 CSV, 고정길이 형식 제공
- 파일 송·수신 시점은 연계 승인 시 설정된 스케줄대로 동작

○ HTTP Header 정보

- HTTP protocol의 Header에 해당 값들을 정의한다.

키	데이터 형식	필수 여부	예시	설명
Accept	string	o	"multipart/form-data"	Content-Type의 값과 <u>동일하게 기재</u>
Content-Type	string	o	"multipart/form-data"	본문의 미디어 타입 기재
cvmis_apikey	string	o	"C919D8A2-61F67E56-12312312-76665842"	연계 서비스가 승인 시 발급되는 api key 사용

- HTTP Header 예제

Accept : multipart/form-data	//Content-Type과 동일
cvmis_apikey : C919D8A2-61F67E56-12312312-76665842	//한국교통안전공단에서 발급해주는 api key
Content-Type : multipart/form-data	//데이터 포맷이 Multipart

○ HTTP Body 정보

- HTTP protocol의 Body에 해당 값들을 정의한다.

키	데이터 형식	필수 여부	예시	설명
content	string		"application/json"	- <u>한국교통안전공단에서 파일을 수신하는 경우 content 에 트랜잭션 ID 값 설정</u> - <u>한국교통안전공단에 파일을 송신하는 경우 설정 하지 않음</u>
file	binary	o	"multipart/form-data"	파일이 여러개일 경우도 같은 키 사용

- 파일 송신 예제 참조

3. 표준 암호·복호화 및 전자서명 절차

□ 표준 암호·복호화 및 전자서명 절차

○ 발급된 인증서를 이용한 메시지 보안

- 메시지 보안을 위해 메시지를 송신하기 전에 한국교통안전공단에서 발급하는 연계 인증서를 이용하여 암호화 및 전자서명을 수행해야한다.
- 반대로 메시지를 수신하면 전자서명을 검증하고 복호화를 수행해야한다.
- 암호화/복호화/전자서명/전자서명검증은 가이드에서 제공하는 예제를 참고한다.

구분	메시지 항목
송신 (⇒)	평문(암호화전) → 암호화 → 전자서명 → BASE64 인코딩
수신 (⇐)	복호화 ← 전자서명 검증 ← BASE64 디코딩 ← 암호화문

□ RESTful 예시

○ HTTP protocol의 Body(content)의 데이터 형식

- HTTP Body의 메시지는 JSON 형식으로 생성한다.
- JSON 객체 최상위에 "data" key가 온다.
- "data"의 값은 배열(array)이며, 배열 안에 요청·응답 데이터가 JSON 객체로 들어간다.
- 암호·복호화는 HTTP body 의 전체를 암호·복호화한다.

○ 요청 메시지 암호화 예시

- 요청 데이터 예시

요청 항목	데이터 형식(길이)	필수여부	설명	비고
IHIDNUM	string(13)	Y	주민번호	- 없이 13자리
NM	string(20)	Y	이름	

- 요청 메시지 예시 (암호화 전)

```
{
  "data":[
    {
      "IHIDNUM":"9104251*****",
      "NM":"홍길동"
    }
  ]
}
```

- 요청 메시지 예시 (암호화 후)

MIIKIAIBATENMAsGCWCGSAFIawQCATCCakAGCSq...

※ HTTP Body의 전체 데이터를 암호화 한다.

○ 응답 메시지 복호화 예시

- 응답 데이터 예시

응답 항목	데이터 형식(길이)	필수여부	설명	비고
VOUCH_SPORT_AT	string(1)	Y	바우처 수급 여부	Y/N
CNTC_RESULT_CODE	string(8)	Y	결과 코드	
CNTC_RESULT_DTLS	string(200)	Y	결과 메시지	

- 응답 메시지 예시 (복호화 전)

MIICOGIBADGCAYQwggGAAgEAMGgwUDELMAkGA1...

※ HTTP Body의 전체 데이터가 암호화되어 있다.

- 응답 메시지 예시 (복호화 후)

```
{
  "data":[
    {
      "VOUCH_SPORT_AT":"Y",
      "CNTC_RESULT_CODE":"00",
      "CNTC_RESULT_DTLS":"성공"
    }
  ]
}
```

□ SOAP 예시

○ HTTP protocol의 Body(content)의 데이터 형식

- HTTP Body의 메시지는 XML 형식으로 생성한다.
- 메시지 구조는 제공된 WSDL 에 정의된 형식대로 송·수신한다.
- 암호·복호화는 HTTP Body 의 SOAP function tag 의 값을 수행한다.

○ 요청 메시지 암호화 예시

- 요청 데이터 예시

요청 항목	데이터 형식(길이)	필수여부	설명	비고
IHIDNUM	string(13)	Y	주민번호	- 없이 13자리
NM	string(20)	Y	이름	

- 요청 메시지 예시 (암호화 전)

```
<?xml version="1.0" encoding="UTF-8"?>
<Envelope xmlns="http://schemas.xmlsoap.org/soap/envelope/">
  <Body>
    <getVoucherYn xmlns="http://car.go.kr/link/vhcle/VoucherYn">
      <IHIDNUM>9104251*****</IHIDNUM>
      <NM>홍길동</NM>
    </getVoucherYn>
  </Body>
</Envelope>
```

- 요청 메시지 예시 (암호화 후)

```
<?xml version="1.0" encoding="UTF-8"?>
<Envelope xmlns="http://schemas.xmlsoap.org/soap/envelope/">
  <Body>
    <getVoucherYn xmlns="http://car.go.kr/link/vhcle/VoucherYn">
      MIIKIAIBATENMAsGCWCGSAFIaWQCATCCaKAGCSq...
    </getVoucherYn>
  </Body>
</Envelope>
```

※ SOAP Function 의 값 전체를 암호화 한다.

○ 응답 메시지 복호화 예시

- 응답 데이터 예시

응답 항목	데이터 형식(길이)	필수여부	설명	비고
VOUCH_SPORT_AT	string(1)	Y	바우처 수급 여부	Y/N
CNTC_RESULT_CODE	string(8)	Y	결과 코드	
CNTC_RESULT_DTLS	string(200)	Y	결과 메시지	

- 응답 메시지 예시 (복호화 전)

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <getVoucherYnResponse xmlns="http://car.go.kr/link/vhcle/VoucherYn">
      MIICOGIBADGCAYQwggGAAGwUELMakGA1...
    </getVoucherYnResponse>
  </soap:Body>
</soap:Envelope>
```

※ SOAP Function 의 값이 암호화 되어 있다.

- 응답 메시지 예시 (복호화 후)

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <getVoucherYnResponse xmlns="http://car.go.kr/link/vhcle/VoucherYn">
      <VOUCH_SPORT_AT>Y</VOUCH_SPORT_AT>
      <CNTC_RESULT_CODE>00</CNTC_RESULT_CODE>
      <CNTC_RESULT_DTLS>성공</CNTC_RESULT_DTLS>
    </getVoucherYnResponse>
  </soap:Body>
</soap:Envelope>
```

4. 연계 개발 샘플

□ 연계 보안 모듈 샘플

- 아래 보안 모듈은 이용기관과 한국교통안전공단간 암호·복호화와 전자서명을 수행하는 모듈
- Java 기반 시스템인 경우 활용 가능한 예시이며, 실제 개발은 각 이용기관이 시스템에 맞도록 자체 개발 필요
 - > Java 8 기반(**Java 1.7이하 버전에서는 다수의 CVE(Common Vulnerabilities and Exposures) 취약점이 발견되어 암호·복호화 모듈로 사용이 불가함**)
 - > Bouncy Castle 라이브러리가 필요하며 컴파일 및 런타임 시 해당 라이브러리의 클래스 경로 설정 필요
 1. <https://mvnrepository.com/artifact/org.bouncycastle/bcprov-jdk18on/1.78.1>
 2. <https://mvnrepository.com/artifact/org.bouncycastle/bcpkix-jdk18on/1.78.1>

```
import org.bouncycastle.asn1.*;
import org.bouncycastle.asn1.x509.AlgorithmIdentifier;
import org.bouncycastle.crypto.PBEParametersGenerator;
import org.bouncycastle.crypto.digests.SHA1Digest;
import org.bouncycastle.crypto.generators.PKCS5S1ParametersGenerator;
import org.bouncycastle.crypto.params.KeyParameter;
import org.bouncycastle.jce.provider.BouncyCastleProvider;
import javax.crypto.*;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import java.io.*;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.security.*;
import java.security.cert.CertificateFactory;
import java.security.spec.PKCS8EncodedKeySpec;
import java.util.Arrays;
import java.util.Base64;
import java.util.List;
public final class KotsaSecurity {
    static {
        Security.addProvider( new BouncyCastleProvider() );
    }
    private static final int BUFFER_SIZE = 4096;
    // 한국교통안전공단 공개키 파일 경로 및 파일명
    private final String publicKeyFile;
    // 연계 승인 후 한국교통안전공단에서 발행한 시스템의 개인키 파일 경로 및 파일명
    private final String privateKeyFile;
    // 인증서 발급 시 설정했던 비밀번호
    private final String privateKeyPassword;

    /**
     * 한국교통안전공단 보안 모듈
     * @param publicKeyFile 한국교통안전공단 공개키
     * @param privateKeyFile 유관기관 개인키(한국교통안전공단에서 발급)
     * @param privateKeyPassword 유관기관 개인키 비밀번호(개인키 발급 시 설정)
     */
    public KotsaSecurity( String publicKeyFile, String privateKeyFile, String
privateKeyPassword ) {
        this.publicKeyFile = publicKeyFile;
        this.privateKeyFile = privateKeyFile;
        this.privateKeyPassword = privateKeyPassword;
    }
}
```

```

/**
 * 메시지를 전송하기 위해 패키징 한다.
 * @param encryptedKey 암호화된 키
 * @param encryptedIv 암호화된 Initial Vector
 * @param signedData 전자서명 데이터
 * @param encryptedData 암호화된 메시지
 * @return 패키지 데이터
 * @throws IOException 메시지 패키징에 실패할 경우 발생
 */
private byte[] dataPacking( byte[] encryptedKey, byte[] encryptedIv, byte[] signedData,
byte[] encryptedData )
    throws IOException {
    ASN1EncodableVector vector = new ASN1EncodableVector();
    vector.add( new DEROctetString(encryptedKey) );
    vector.add( new DEROctetString(encryptedIv) );
    vector.add( new DEROctetString(signedData) );
    vector.add( new DEROctetString(encryptedData) );
    return new DERSequence( vector ).getEncoded();
}

/**
 * 파일을 전송하기 위해 패키징하고 결과를 새로운 파일에 기록한다.
 * @param encryptedKey 암호화된 키
 * @param encryptedIv 암호화된 Initial Vector
 * @param signedData 전자서명 데이터
 * @param inputFile 암호화된 파일
 * @param outputFile 패키징된 파일
 * @throws Exception 파일 패키징에 실패할 경우 발생
 */
private void dataPacking(
    byte[] encryptedKey
    , byte[] encryptedIv
    , byte[] signedData
    , File inputFile
    , File outputFile ) throws Exception {
    try( FileInputStream fis = new FileInputStream( inputFile );
        DataOutputStream dos = new DataOutputStream( new
FileOutputStream(outputFile) ) ) {
        dos.writeInt( encryptedKey.length );
        dos.write( encryptedKey );
        dos.writeInt( encryptedIv.length );
        dos.write( encryptedIv );
        dos.writeInt( signedData.length );
        dos.write( signedData );
        byte[] buffer = new byte[ BUFFER_SIZE ];
        int readSize;
        while( (readSize = fis.read(buffer)) > 0 ) {
            dos.write( buffer, 0, readSize );
        }
        dos.flush();
    }
}

/**
 * 수신된 메시지의 패키지를 해제한다.
 * @param packagedData 패키징된 데이터
 * @return 해제된 데이터(암호화된 키, 암호화된 Initial Vector, 전자서명 데이터, 암호화된
메시지)
 * @throws IOException 패키징된 데이터에 문제가 있을 경우 발생
 */
private List<byte[]> dataUnpacking( byte[] packagedData ) throws IOException {
    try( ASN1InputStream is = new ASN1InputStream(packagedData) ) {
        ASN1Sequence asn1Sequence = (ASN1Sequence)is.readObject();
        return Arrays.asList(
            ((DEROctetString)asn1Sequence.getObjectAt( 0 )).getOctets()
            , ((DEROctetString)asn1Sequence.getObjectAt( 1 )).getOctets()
            , ((DEROctetString)asn1Sequence.getObjectAt( 2 )).getOctets()
            , ((DEROctetString)asn1Sequence.getObjectAt( 3 )).getOctets()
        );
    }
}

```



```

    }
}

/**
 * Base64 로 인코딩된 데이터를 디코딩한다.
 * @param data Base64 로 인코딩된 데이터
 * @return 디코딩 데이터
 */
private byte[] decode( byte[] data ) {
    return Base64.getDecoder().decode( data );
}

/**
 * 개인키를 이용해서 암호화된 데이터를 복호화 한다.
 * @param privateKey 해당 기관의 개인키
 * @param encryptedData 암호화된 데이터
 * @return 복호화된 데이터
 * @throws Exception 데이터 복호화에 실패할 경우 발생
 */
private byte[] decryptByPrivateKey( PrivateKey privateKey, byte[] encryptedData ) throws
Exception {
    Cipher cipher = Cipher.getInstance( "RSA" );
    cipher.init( Cipher.DECRYPT_MODE, privateKey );
    return cipher.doFinal( encryptedData );
}

/**
 * 메시지 복호화
 * @param key 복호화에 사용될 키
 * @param iv Initial Vector
 * @param encryptedMessage 암호화된 메시지
 * @return 복호화 메시지
 * @throws Exception 메시지 복호화가 실패할 경우 발생
 */
private byte[] decryptByKey( SecretKey key, IvParameterSpec iv, byte[]
encryptedMessage ) throws Exception {
    Cipher cipher = Cipher.getInstance( "SEED/CBC/PKCS5Padding", "BC" );
    cipher.init( Cipher.DECRYPT_MODE, key, iv );
    return cipher.doFinal( encryptedMessage );
}

/**
 * 파일 복호화
 * @param key 복호화에 사용될 키
 * @param iv Initial Vector
 * @param inputFile 암호화 파일
 * @param outputFile 복호화 파일
 * @throws Exception 파일 복호화에 실패할 경우 발생
 */
private void decryptByKey( SecretKey key, IvParameterSpec iv, File inputFile, File
outputFile ) throws Exception {
    Cipher cipher = Cipher.getInstance( "SEED/CBC/PKCS5Padding", "BC" );
    cipher.init( Cipher.DECRYPT_MODE, key, iv );
    try( CipherInputStream cis = new CipherInputStream( new FileInputStream(inputFile),
cipher );
        FileOutputStream fos = new FileOutputStream( outputFile ) ) {
        byte[] buffer = new byte[ BUFFER_SIZE ];
        int readSize;
        while( (readSize = cis.read(buffer)) >= 0 ) {
            fos.write( buffer, 0, readSize );
        }
        fos.flush();
    }
}

/**
 * 바이너리 데이터를 Base64 로 인코딩 한다.
 * @param data 바이너리 데이터
 * @return Base64 인코딩 데이터

```

```

    */
    private byte[] encode( byte[] data ) {
        return Base64.getEncoder().encode( data );
    }

    /**
     * 공개키를 이용해서 데이터를 암호화한다.
     * @param publicKey 상대 기관의 공개키
     * @param data 암호화 대상 데이터
     * @return 암호화된 데이터
     * @throws Exception 데이터 암호화에 실패할 경우 발생
     */
    private byte[] encryptByPublicKey( PublicKey publicKey, byte[] data ) throws Exception {
        Cipher cipher = Cipher.getInstance( "RSA" );
        cipher.init( Cipher.ENCRYPT_MODE, publicKey );
        return cipher.doFinal( data );
    }

    /**
     * 메시지 암호화
     * @param key 암호화에 사용될 키
     * @param iv Initial Vector
     * @param message 평문 메시지
     * @return 암호화된 메시지
     * @throws Exception 메시지 암호화가 실패할 경우 발생
     */
    private byte[] encryptByKey( SecretKey key, IvParameterSpec iv, String message )
    throws Exception {
        Cipher cipher = Cipher.getInstance( "SEED/CBC/PKCS5Padding", "BC" );
        cipher.init( Cipher.ENCRYPT_MODE, key, iv );
        return cipher.doFinal( message.getBytes() );
    }

    /**
     * 파일 암호화
     * @param key 암호화에 사용될 키
     * @param iv Initial Vector
     * @param inputFile 평문 파일
     * @param outputFile 암호화된 파일
     * @throws Exception 파일 암호화가 실패할 경우 발생
     */
    private void encryptByKey( SecretKey key, IvParameterSpec iv, File inputFile, File
    outputFile ) throws Exception {
        Cipher cipher = Cipher.getInstance( "SEED/CBC/PKCS5Padding", "BC" );
        cipher.init( Cipher.ENCRYPT_MODE, key, iv );
        try( FileInputStream fis = new FileInputStream( inputFile );
            CipherOutputStream cos = new CipherOutputStream( new
            FileOutputStream( outputFile ), cipher ) ) {
            byte[] buffer = new byte[ BUFFER_SIZE ];
            int readSize;
            while( (readSize = fis.read(buffer)) > 0 ) {
                cos.write( buffer, 0, readSize );
            }
            cos.flush();
        }
    }

    /**
     * 특정 파일의 Hash 를 추출한다.
     * @param file 대상 파일
     * @return hash value
     * @throws Exception Hash 추출에 실패할 경우 발생
     */
    private byte[] extractFileHash( File file ) throws Exception {
        MessageDigest sha256 = MessageDigest.getInstance( "SHA-256" );
        byte[] hash;
        try( DigestInputStream dis = new DigestInputStream( new FileInputStream(file),
        sha256 ) ) {
            byte[] buffer = new byte[ BUFFER_SIZE ];

```

```

        while( dis.read(buffer) > 0 ) {}
        hash = sha256.digest();
    }
    return hash;
}

/**
 * 데이터 암호화에 필요한 Initial Vector 생성
 * @return 생성된 IV
 */
private IvParameterSpec generateIv() {
    byte[] ivBytes = new byte[ 16 ];
    new SecureRandom().nextBytes( ivBytes );
    return new IvParameterSpec( ivBytes );
}

/**
 * 데이터 암호화를 위한 키 생성
 * @return 생성된 키
 * @throws Exception 키 생성에 실패할 경우 발생
 */
private SecretKey generateKey() throws Exception {
    KeyGenerator keyGenerator = KeyGenerator.getInstance( "SEED", "BC" );
    keyGenerator.init( 128 );
    return keyGenerator.generateKey();
}

/**
 * 전자서명
 * @param privateKey 개인키(이용기관에서 보유한 개인키)
 * @param encryptedData 암호화된 메시지 데이터
 * @return 서명 데이터
 * @throws Exception 전자서명에 실패할 경우 발생
 */
private byte[] sign( PrivateKey privateKey, byte[] encryptedData ) throws Exception {
    Signature sig = Signature.getInstance( "SHA256withRSA" );
    sig.initSign( privateKey );
    sig.update( encryptedData );
    return sig.sign();
}

/**
 * 전자서명
 * @param privateKey 개인키(이용기관에서 보유한 개인키)
 * @param encryptedFile 암호화된 파일
 * @return 서명 데이터
 * @throws Exception 전자서명에 실패할 경우 발생
 */
private byte[] sign( PrivateKey privateKey, File encryptedFile ) throws Exception {
    Signature sig = Signature.getInstance( "SHA256withRSA" );
    sig.initSign( privateKey );
    sig.update( extractFileHash(encryptedFile) );
    return sig.sign();
}

/**
 * 전자서명 검증
 * @param publicKey 공개키(한국교통안전공단 공개키)
 * @param verifyData 검증 대상 데이터
 * @param signData 서명 데이터
 * @return true - 검증성공, false - 검증실패
 * @throws Exception 전자서명 검증에 실패할 경우 발생
 */
private boolean verify( PublicKey publicKey, byte[] verifyData, byte[] signData ) throws
Exception {
    Signature sig = Signature.getInstance( "SHA256withRSA" );
    sig.initVerify( publicKey );
    sig.update( verifyData );
    return sig.verify( signData );
}

```

```

    }

    /**
     * 개인키를 가져온다.
     * @return 복호화된 개인키
     * @throws Exception 개인키를 가져오지 못 할 경우 발생
     */
    private PrivateKey readPrivateKey() throws Exception {
        try( ASN1InputStream is = new ASN1InputStream(
            Files.newInputStream(Paths.get(privateKeyFile))) ) {
            ASN1Sequence asn1Sequence = (ASN1Sequence)is.readObject();
            AlgorithmIdentifier algorithmIdentifier = AlgorithmIdentifier.getInstance(
                asn1Sequence.getObjectAt(0) );
            ASN1OctetString data = ASN1OctetString.getInstance(
                asn1Sequence.getObjectAt(1) );
            ASN1Sequence asn1Sequence2 =
                (ASN1Sequence)algorithmIdentifier.getParameters();
            DEROctetString salt = (DEROctetString)asn1Sequence2.getObjectAt( 0 );
            ASN1Integer ic = (ASN1Integer)asn1Sequence2.getObjectAt( 1 );
            PKCS5S1ParametersGenerator generator = new PKCS5S1ParametersGenerator(
                new SHA1Digest() );
            generator.init(
                PBEPParametersGenerator.PKCS5PasswordToBytes(privateKeyPassword.toCharArray())
                    , salt.getOctets()
                    , ic.getValue().intValue()
            );
            byte[] derivedKey =
                ((KeyParameter)generator.generateDerivedParameters(20*8)).getKey();
            byte[] keyData = new byte[ 16 ];
            System.arraycopy( derivedKey, 0, keyData, 0, 16 );
            byte[] digestBytes = new byte[ 4 ];
            System.arraycopy( derivedKey, 16, digestBytes, 0, 4 );
            MessageDigest md = MessageDigest.getInstance( "SHA-1" );
            md.reset();
            md.update( digestBytes );
            byte[] div = md.digest();
            byte[] iv = new byte[ 16 ];
            System.arraycopy( div, 0, iv, 0, 16 );
            IvParameterSpec ivSpec = new IvParameterSpec( iv );
            SecretKeySpec keySpec = new SecretKeySpec( keyData, "SEED" );
            Cipher cipher = Cipher.getInstance( "SEED/CBC/PKCS5Padding", "BC" );
            cipher.init( Cipher.DECRYPT_MODE, keySpec, ivSpec );
            byte[] decryptedKey = cipher.doFinal( data.getOctets() );
            return KeyFactory.getInstance( "RSA", "BC" )
                .generatePrivate( new PKCS8EncodedKeySpec(decryptedKey) );
        }
    }

    /**
     * 공개키를 가져온다.
     * @return 공개키
     * @throws Exception 공개키 가져오는데 실패할 경우 발생
     */
    private PublicKey readPublicKey() throws Exception {
        try( InputStream is = Files.newInputStream(Paths.get(publicKeyFile)) ) {
            return CertificateFactory.getInstance( "X.509"
                ).generateCertificate(is).getPublicKey();
        }
    }

    /**
     * 파일을 암호화 한다.
     * @param originFile 원문 파일
     * @param encryptedFile 암호화된 송신 대상 파일
     * @throws Exception 파일 암호화에 실패할 경우 발생
     */
    public void fileEncrypt( File originFile, File encryptedFile ) throws Exception {
        final PublicKey kotsaPublicKey = readPublicKey();
    }

```

```

        final PrivateKey selfPrivateKey = readPrivateKey();
        // 키 생성
        SecretKey key = generateKey();
        // Initial Vector 생성
        IvParameterSpec iv = generateIv();
        // 암호화 임시파일
        File encryptedTempFile = new File( originFile.getAbsolutePath().getParent()
            , originFile.getName() + ".enc" );
        if( !encryptedTempFile.exists() && !encryptedTempFile.isFile() &&
!encryptedTempFile.createNewFile() )
            throw new IllegalStateException( "암호화를 위한 임시파일을 생성할 수 없습니다." );
        // 파일 암호화
        encryptByKey( key, iv, originFile, encryptedTempFile );
        // 전자서명
        byte[] signedData = sign( selfPrivateKey, encryptedTempFile );
        // 키 암호화
        byte[] encryptedKey = encryptByPublicKey( kotsaPublicKey, key.getEncoded() );
        // Initial Vector 암호화
        byte[] encryptedIv = encryptByPublicKey( kotsaPublicKey, iv.getIV() );
        // 데이터 패키징
        dataPacking( encryptedKey, encryptedIv, signedData, encryptedTempFile,
encryptedFile );
        // 암호화 임시파일 삭제
        if( !encryptedTempFile.delete() )
            throw new IllegalStateException( "암호화를 위해 생성한 임시파일을 삭제할 수
없습니다." );
    }

    /**
     * 파일을 복호화 한다.
     * @param encryptedFile 암호화된 수신 파일
     * @param originFile 복호화된 원문 파일
     * @throws Exception 파일 복호화에 실패할 경우 발생
     */
    public void fileDecrypt( File encryptedFile, File originFile ) throws Exception {
        final PublicKey kotsaPublicKey = readPublicKey();
        final PrivateKey selfPrivateKey = readPrivateKey();
        if( !originFile.exists() && !originFile.isFile() && !originFile.createNewFile() )
            throw new IllegalStateException( "복호화 파일을 생성할 수 없습니다." );
        try( DataInputStream dis = new DataInputStream( new FileInputStream(encryptedFile)
) ) {
            byte[] encryptedKey = new byte[ dis.readInt() ];
            dis.readFully( encryptedKey );
            byte[] encryptedIv = new byte[ dis.readInt() ];
            dis.readFully( encryptedIv );
            byte[] signedData = new byte[ dis.readInt() ];
            dis.readFully( signedData );
            File decryptTempFile = new File( encryptedFile.getAbsolutePath().getParent()
                , encryptedFile.getName() + ".enc" );
            if( !decryptTempFile.createNewFile() )
                throw new IllegalStateException( "복호화를 위한 임시파일을 생성할 수 없습니다."
);
            try( FileOutputStream fos = new FileOutputStream( decryptTempFile ) ) {
                byte[] buffer = new byte[ BUFFER_SIZE ];
                int readSize;
                while( (readSize = dis.read( buffer )) >= 0 ) {
                    fos.write( buffer, 0, readSize );
                }
                fos.flush();
            }
            byte[] hash = extractFileHash( decryptTempFile );
            // 서명 검증
            boolean verify = verify( kotsaPublicKey, hash, signedData );
            // verify 가 false 이면 서명 검증이 실패되었기 때문에 다음 프로세스를 수행하지
않는다.
            if( !verify ) throw new Exception( "전자서명 검증에 실패하였습니다." );
            // 키 복호화
            SecretKey decryptedKey =
                new SecretKeySpec( decryptByPrivateKey( selfPrivateKey, encryptedKey ),

```

```

"SEED" );
    // Initial Vector 복호화
    IvParameterSpec decryptedIv =
        new IvParameterSpec( decryptByPrivateKey( selfPrivateKey, encryptedIv )
);
    // 메시지 복호화
    decryptByKey( decryptedKey, decryptedIv, decryptTempFile, originFile );
    if( !decryptTempFile.delete() )
        throw new IllegalStateException( "복호화를 위해 생성한 임시파일을 삭제할 수
없습니다." );
}

/**
 * 실시간 연계 수신된 데이터를 복호화 한다.
 * @param encryptedData 암호화된 데이터
 * 예) MIIC0glBADGCAYQwggGAAGeAMGgwUDELMAkGA1...
 * @return 복호화된 원문 메시지
 * @throws Exception 메시지 복호화에 실패할 경우 발생
 */
public String realtimeDecrypt( String encryptedData ) throws Exception {
    final PublicKey kotsaPublicKey = readPublicKey();
    final PrivateKey selfPrivateKey = readPrivateKey();
    // 패키지 데이터 해체
    // 첫번째: 암호화된 키
    // 두번째: 암호화된 Initial Vector
    // 세번째: 전자서명 데이터
    // 네번째: 암호화된 메시지
    List<byte[]> unpackedData = dataUnpacking( decode(encryptedData.getBytes()) );
    // 서명 검증
    boolean verify = verify( kotsaPublicKey, unpackedData.get(3), unpackedData.get(2) );
    // verify 가 false 이면 서명 검증이 실패되었기 때문에 다음 프로세스를 수행하지 않는다.
    if( !verify ) throw new Exception( "전자서명 검증에 실패하였습니다." );
    // 키 복호화
    SecretKey decryptedKey =
        new SecretKeySpec( decryptByPrivateKey(selfPrivateKey,
unpackedData.get(0)), "SEED" );
    // Initial Vector 복호화
    IvParameterSpec decryptedIv =
        new IvParameterSpec( decryptByPrivateKey(selfPrivateKey,
unpackedData.get(1)) );
    // 메시지 복호화
    return new String( decryptByKey(decryptedKey, decryptedIv, unpackedData.get(3)) );
}

/**
 * 실시간 요청 전송을 위해 암호화 처리
 * @param requestData 요청 전문 메시지
 * 예) <IHIDNUM>9104251*****</IHIDNUM><NM>홍길동</NM>
 * @throws Exception 암호화 처리에 실패할 경우 발생
 */
public String realtimeEncrypt( String requestData ) throws Exception {
    final PublicKey kotsaPublicKey = readPublicKey();
    final PrivateKey selfPrivateKey = readPrivateKey();
    // 키 생성
    SecretKey key = generateKey();
    // Initial Vector 생성
    IvParameterSpec iv = generateIv();
    // 메시지 암호화
    byte[] encryptedMessage = encryptByKey( key, iv, requestData );
    // 전자 서명
    byte[] signedData = sign( selfPrivateKey, encryptedMessage );
    // 키 암호화
    byte[] encryptedKey = encryptByPublicKey( kotsaPublicKey, key.getEncoded() );
    // Initial Vector 암호화
    byte[] encryptedIv = encryptByPublicKey( kotsaPublicKey, iv.getIV() );
    // 데이터 패키징
    byte[] packagedData = dataPacking( encryptedKey, encryptedIv, signedData,
encryptedMessage );
    return new String( encode(packagedData) );
}
}

```

□ 실시간 암호·복호화 샘플

- 아래 개발 예제는 연계 보안 모듈을 사용한 실시간 암호·복호화 샘플
- 메시지 형식별(JSON, XML) 파싱 부분과 암호화된 콘텐츠 영역을 추출하는 부분은 이용기관이 시스템에 맞도록 자체 개발 필요

```
class KotsaRealtimeSample {
    public static void main( String ... args ) throws Exception {
        String originMessage = "실시간 원문 데이터";
        System.out.println( "origin message: "+ originMessage );
        KotsaSecurity kotsa = new KotsaSecurity( "./certs/signCert.der",
        "./certs/signPri.key", "p@ssWord" );

        // 메시지 암호화
        String encryptMessage = kotsa.realtimeEncrypt( originMessage );
        System.out.println( "encrypt message: "+ encryptMessage );

        // 메시지 복호화
        String decryptMessage = kotsa.realtimeDecrypt( encryptMessage );
        System.out.println( "decrypt message: "+ decryptMessage );
    }
}
```

□ 파일 암호·복호화 샘플

- 아래 개발 예제는 연계 보안 모듈을 사용한 파일 암호·복호화 샘플
- 한국교통안전공단으로 암호화된 파일을 송신하는 예제는 아래 배치 연계 송신 샘플 참조

```
import java.io.File;
class KotsaBatchTest {
    public static void main( String ... args ) throws Exception {
        KotsaSecurity kotsa = new KotsaSecurity( "./certs/signCert.der",
        "./certs/signPri.key", "p@ssWord" );

        // 파일 암호화
        File originFile = new File( "/path/to/원본파일" );
        kotsa.fileEncrypt( originFile, new File( "/path/to/암호화된파일" ) );

        // 파일 복호화
        File encryptedFile = new File( "/path/to/암호화된파일" );
        kotsa.fileDecrypt( encryptedFile, new File( "/path/to/복호화된파일" ) );
    }
}
```


□ 배치 연계 송신 샘플

- 이용기관이 한국교통안전공단에 파일을 송신하는 개발 샘플
- 이용기관이 한국교통안전공단에서 파일을 수신하는 부분은 각 이용기관이 시스템에 맞도록 자체 개발 필요
- 아래 개발 샘플은 Java 기반 시스템인 경우 활용 가능한 예시이며, 실제 개발은 각 이용기관이 시스템에 맞도록 자체 개발 필요

> Java 8 기반

> Apache Component 라이브러리가 필요하며 컴파일 및 런타임 시 해당 라이브러리의 클래스 경로 설정 필요

1. <https://mvnrepository.com/artifact/org.apache.httpcomponents.core5/httpcore5/5.2.4>
2. <https://mvnrepository.com/artifact/org.apache.httpcomponents.core5/httpcore5-h2/5.2.4>
3. <https://mvnrepository.com/artifact/org.apache.httpcomponents.client5/httpclient5/5.3.1>

```
import org.apache.hc.client5.http.ClientProtocolException;
import org.apache.hc.client5.http.classic.methods.HttpPost;
import org.apache.hc.client5.http.entity.mime.MultipartEntityBuilder;
import org.apache.hc.client5.http.impl.classic.CloseableHttpClient;
import org.apache.hc.client5.http.impl.classic.HttpClients;
import org.apache.hc.core5.http.ContentType;
import org.apache.hc.core5.http.HttpEntity;
import org.apache.hc.core5.http.io.HttpClientResponseHandler;
import org.apache.hc.core5.http.io.entity.EntityUtils;
import java.io.File;
import java.io.IOException;

public class FileSendTest {
    public static void main( String ... args ) {
        // 해당 URL 은 한국교통안전공단 테스트 서버이며 실 운용 시 변경될 수 있음
        String url = "http://210.91.35.175:28080/hub/kotsa";
        String jsonData = "{\"data\":[ ]}";
        // TODO: 전송할 파일 경로 및 파일 설정
        File file1 = new File( "/path/to/file1.txt" );
        // TODO: 파일이 두 개 이상일 경우 설정
        File file2 = new File( "/path/to/file2.txt" );
        try( CloseableHttpClient client = HttpClients.createDefault() ) {
            MultipartEntityBuilder builder = MultipartEntityBuilder.create();
            builder.addTextBody( "content", jsonData, ContentType.APPLICATION_JSON );
            builder.addBinaryBody( "file", file1, ContentType.MULTIPART_FORM_DATA,
file1.getName() );
            // TODO: 파일이 한 개 일 경우 설정하지 않음
            builder.addBinaryBody( "file", file2, ContentType.MULTIPART_FORM_DATA,
file2.getName() );
            HttpPost httpPost = new HttpPost( url );
            // 한국교통안전공단에서 제공한 API KEY 값 설정
            httpPost.setHeader( "cvmis_apikey", "APIKEY" );
            httpPost.setEntity( builder.build() );
            HttpClientResponseHandler<String> responseHandler = response -> {
                int status = response.getStatusCode();
                if( status >= 200 && status < 300 ) {
                    HttpEntity entity = response.getEntity();
                    return entity != null ? EntityUtils.toString( entity ) : null;
                } else
                    throw new ClientProtocolException( "요청이 실패하였습니다." );
            };
            String responseBody = client.execute( httpPost, responseHandler );
            // TODO: 응답 본문 처리
        } catch( IOException ex ) {
            ex.printStackTrace();
            // TODO: 예러 처리
        }
    }
}
```